

1. For each description (a)-(j) specify the matching term from the list of terms. 10 points

- | | |
|------------------------|--------------------------|
| 1. AVL tree | 16. Height |
| 2. Binary tree | 17. In-order |
| 3. Coalescing | 18. Internal path length |
| 4. Compacting | 19. LIFO |
| 5. Complete | 20. Leaf |
| 6. Complexity class | 21. Level |
| 7. Dominant term | 22. Level-order |
| 8. Equilibrium | 23. Pre-order |
| 9. Extended | 24. Post-order |
| 10. FIFO | 25. Queue |
| 11. Fragmentation | 26. Recurrence relation |
| 12. Frame pointer | 27. Root |
| 13. Free list | 28. Roving pointer |
| 14. Garbage collection | 29. Stack |
| 15. Heap | 30. Trie |
| | 31. Two-three tree |

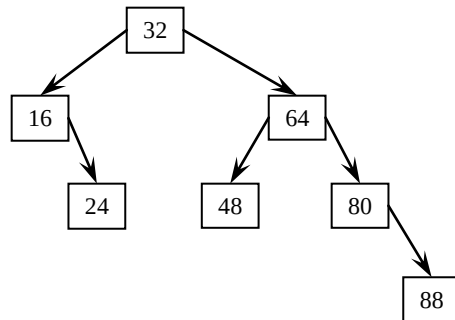
- a) ____: An alternative name for a stack.
- b) ____: A name for a node in a tree that has no descendants.
- c) ____: Either the empty tree or a node that has left and right subtrees that are binary trees.
- d) ____: A traversal order in which we traverse a tree left to right starting at the top and moving to the bottom.
- e) ____: A complete binary tree having values at the nodes such that the values are arranged in descending order along any path from the root toward the leaves.
- f) ____: A tree that is insensitive to the order of insertion of the keys, but is sensitive to clustering in the spelling of the prefixes of the keys.
- g) ____: Occurs when large memory blocks are split to satisfy requests for more storage.
- h) ____: Expresses the cost of solving a whole problem into the component costs of solving the sub-problems and combining their solutions
- i) ____: A binary tree that has its empty binary subtrees explicitly represented
- j) ____: A binary tree with leaves on either a single level or on two adjacent levels such that the leaves on the bottommost level are placed as far left as possible.

2. Short answer questions (20 points)

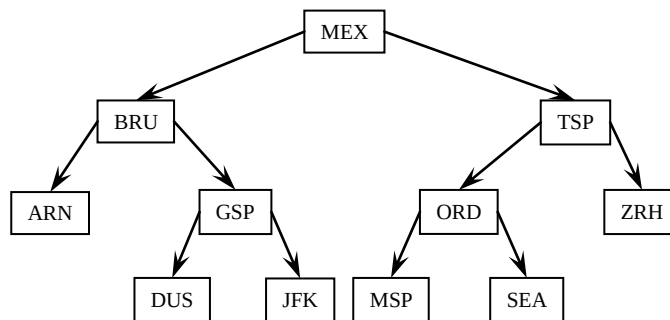
- 2.1. True or False: The Queue ADT can be used to process nested structures such as checking for balanced parenthesis.
- 2.2. True or False: An inorder traversal always processes the elements of a binary search tree in the same order, regardless of the order in which the elements were inserted.
- 2.3. True or False: An operating system makes extensive use of stacks to hold ordered lists of processes, network packets, I/O waiting lists, and print buffers.
- 2.4. For a complete binary tree with n nodes, how many nodes are leaf nodes?
- 2.5. If search for each of the keys, K , in a binary search tree of n nodes is equally likely, and if E is the external path length of the tree, then the number of comparisons required by an average unsuccessful search is equal to (hint: a function of E and n):
- 2.6. How many nodes are in a complete binary tree with n levels in which the bottom level is full. (A tree with one node has one level).
- 2.7. For a complete binary tree with n nodes, how many levels does the tree have?
- 2.8. Give the PostOrder traversal of the tree shown in problem 3.1 (without considering the insertion).

3. Trees. (30 points)

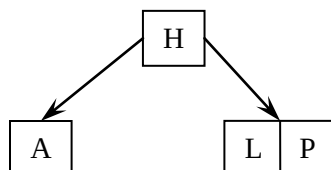
3.1. Insert 20 into the following AVL tree.



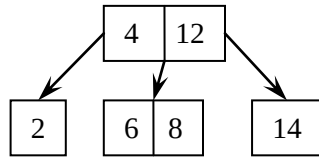
3.2. Insert SAN into the following AVL tree.



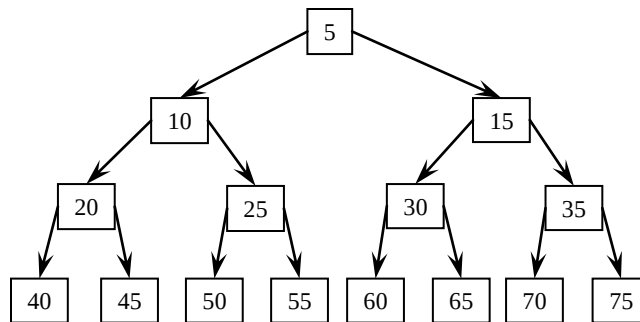
3.3. Insert J into the following Two-three tree. Draw the final tree.



3.4. Insert 5 into the following Two-three tree. Draw the final tree.



3.5 The following is a heap in which the highest priority item has the lowest value. Insert a new item with value 15 into the heap.



3.6. The following priority queue is implemented as a heap using a sequential representation. The item with the lowest number has the highest priority. Show the heap after the high priority item is removed from the queue.

7	10	16	40	43	51	21	48	45	49
---	----	----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--	--	--

4. Algorithm design. (20 points)

- 4.1. Determine the runtime complexity class to execute the algorithm method. Assume the code executes in an amount of time that does not depend on n .

```
void method(int n) {  
    int i = n;  
    while (i > 0) {  
        /* the code */  
        i = i/2;  
    }  
}
```

- 4.2. Consider an application that utilizes a list ADT. The application makes approximately an equal number of calls to access an item in the list at a random location, insert an item into a random location, and remove an item from a random location. You determine that the list is usually very large and the random locations are equally likely to be located at any position in the list for most of the calls. However, there are occasional occurrences where a large number of insertions and deletions are performed in sequentially increasing locations. Your initial implementation utilizes a sequential array (which always has sufficient room), but after evaluating the run time you determine you need to reduce the total time spent on accesses, insertions, and deletions. Briefly **describe** an alternative implementation for the list ADT, and **fill in the table** with the average time expected to be spent on accesses, insertions, and deletions (in terms of big-O notation for a list of average size n) for your representation and for the sequential implementation.

Operation	Sequential representation	Your representation
Access		
Insertion		
Deletion		

- 4.3. During a job interview, the following question is posed to two applicants: Describe an algorithm that when given a pointer to a node, **N**, in a binary search tree (BST), returns a pointer to the successor (i.e., the node with the next largest key) or NULL if the node contains the largest key in the tree. The applicants recall that their data structures instructor, Professor Alf Witt, posted a homework solution to this very question (the solution is listed below). Mr. Leff Wright offers this solution in answer to the question while Ms. Daisee Chayne points out that this solution does not solve the problem in general, though this algorithm is useful when programming a BST. Ms. Chayne is hired because she gives a small counterexample, showing a case when the successor is not found, and gives a brief sketch for an algorithm that is always correct. **Give a small counterexample.** (The algorithm to always find the successor in challenging and is a topic for an advanced algorithms class.)

```
bst_node_t * Successor(bst_node_t *N) {  
    N = N->right;  
    if (N != NULL)  
        while (N->left != NULL)  
            N = N->left;  
    return N;  
}
```

5. Binary search tree with linked representation. (20 points)

Write a **recursive** function that takes a pointer to the root node in a binary search tree and returns a ternary distribution on the number of keys stored in the tree. Assume all key values are positive integers. Two of the input parameters to the function must be **thresh_low** and **thresh_high**. The function must return the values of three counters: one counter holds a count of the number of nodes with key value $\leq$ **thresh_low**; the second counter holds a count of the number of nodes with a key value $\geq$ **thresh_high**; the third counter holds a count of the number of nodes whose key value **K** is: **thresh_low** < **K** < **thresh_high**. No credit will be given for an iterative solution, or a solution that uses global or static variables. For simplicity, you can assume the counters are set to zero before the function is called.